

Communication in Distributed Architecture

Optimizing Point-to-Point Data Transfer Throughput

Eddie Odiwuor Ogendo

January 18, 2026

Abstract

This research investigates the upper limits of data transfer rates between two point nodes. The primary objective is to evaluate the impact of different communication protocols, message sizes, and Modular Component Architecture (MCA) tuning on total bandwidth and latency.

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Motivation	2
2	Technical Background	2
2.1	The OpenMPI Architecture	2
2.2	Interconnect Technologies	2
2.3	Communication Protocols	2
3	Experimental Methodology	2
3.1	Hardware Setup	2
3.2	Software Environment	2
3.3	Benchmark Design	3
4	Optimization Strategies	3
4.1	Non-Blocking Communication	3
4.2	Remote Memory Access (RMA)	3
4.3	MCA Parameter Tuning	3
5	Results and Analysis	3
5.1	Bandwidth Analysis	3
5.2	Mathematical Performance Model	3
5.3	Comparison of Protocols	3
6	Conclusion	3

1 Introduction

Detailed overview of the data movement problem in distributed systems.

1.1 Problem Statement

Focus on the "Two-Node Bottleneck" and theoretical peak vs. achieved performance.

1.2 Motivation

Why moving data at wire-speed is critical for [insert your specific use case, e.g., CFD, Climate Modeling, or Big Data].

2 Technical Background

2.1 The OpenMPI Architecture

Briefly explain the Modular Component Architecture (MCA) and the Point-to-Point Management Layer (PML).

2.2 Interconnect Technologies

Technical specifications of the hardware used (e.g., InfiniBand EDR, 100GbE, RoCE).

2.3 Communication Protocols

Distinguish between the Eager and Rendezvous protocols and their impact on throughput.

3 Experimental Methodology

3.1 Hardware Setup

Describe the CPU, RAM, and NIC (Network Interface Card) of the two nodes.

3.2 Software Environment

The software stack was selected to ensure high-performance execution and minimal overhead during point-to-point data transfers. The environment is detailed as follows:

- **Operating System:** The experiments were conducted on a Windows Subsystem for Linux running Ubuntu 24.04.3 LTS with the **Kernel version 6.6.87.2-microsoft-standard-WSL2**. To ensure maximum throughput, we verified that `sysctl` parameters for TCP/IP (such as `net.core.rmem_max` and `net.core.wmem_max`) were tuned to support large window sizes for high-bandwidth networks.
- **MPI Implementation:** We utilized **OpenMPI v5.0.9** (stable). The library was built with UCX (Unified Communication X) support to leverage RDMA capabilities for the underlying InfiniBand/RoCE interconnect. The Modular Component Architecture (MCA) was configured at runtime to prioritize the `btl_openib` and `pml_ucx` components for optimal hardware saturation.
- **Compiler and Flags:** All C source files were compiled using the **GCC 14.2.0** suite. To achieve "wire-speed" performance and ensure the CPU does not become the bottleneck during data packing/unpacking, the following optimization flags were applied:

- `-O3`: Enables aggressive code optimizations including loop vectorization.
- `-march=native`: Instructs the compiler to generate instructions specific to the host CPU architecture (e.g., AVX-512).
- `-mtune=native`: Optimizes the generated code for the local processor’s pipeline.
- `-ffast-math`: Relaxes some strict IEEE floating-point compliance to further speed up arithmetic during data processing.

3.3 Benchmark Design

Description of the Ping-Pong benchmark logic using `MPI_Send` and `MPI_Recv`.

4 Optimization Strategies

4.1 Non-Blocking Communication

Implementation of `MPI_Isend` and `MPI_Irecv` to hide latency.

4.2 Remote Memory Access (RMA)

One-sided communication using `MPI_Put` and `MPI_Get`.

4.3 MCA Parameter Tuning

Analysis of specific runtime flags like `btl_openib_eager_limit` and `pml_ucx`.

5 Results and Analysis

5.1 Bandwidth Analysis

Present the bandwidth curve relative to message size.

5.2 Mathematical Performance Model

The observed bandwidth $B(L)$ for a message of size L is modeled as:

$$B(L) = \frac{L}{\alpha + \beta L}$$

Where α represents latency and β represents the reciprocal of the peak bandwidth.

5.3 Comparison of Protocols

Graph comparing Blocking vs. Non-blocking vs. RMA.

6 Conclusion

Summary of the most effective configuration and final throughput achieved.

References

- [1] The Open MPI Project. *Open MPI: A High Performance Message Passing Library*.
<https://www.open-mpi.org/>
- [2] MPI Basics by Tom Nurkkalla. *Introduction to distributed computing with MPI*.
<https://www.youtube.com/watch?v=c0C9mQaxsD4>
- [3] Introduction to Parallel Programming using MPI. <https://arcca.github.io/intro-mpi/>